

Programação de Computadores (CI208)

Lista de Exercícios

Primeiro semestre de 2025

Funções

IMPORTANTE: A PARTIR DE AGORA TENHA USAR O MÁXIMO DE FUNÇÕES POSSÍVEIS

1. **(Fácil)** Faça uma função que receba como parâmetros dois números inteiros não nulos e retorne *true* se um for o contrário do outro e *false* em caso contrário. Teste sua função usando este código:

```
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    if (contrario(n, m) {
        cout << n << " eh o contrario de " << m << "\n";
    } else {
        cout << n << "\" nao eh o contrario de \"" << m << "\n";
    }
    return 0;
}
```

2. **(Fácil)** Faça uma função que receba como parâmetro um número inteiro e teste se ele é um número binário. Se ele for binário, imprima *sim* senão imprima *nao*. Teste sua função usando este código:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    if (ehBinario(n)) {
        cout << "sim\n";
    } else {
        cout << "nao\n";
    }
    return 0;
}
```

3. **(Fácil)** Faça uma função que receba como parâmetro um número inteiro e retorne *true* se ele for primo e *false* em caso contrário. Teste sua função usando o código abaixo, que imprime todos os primos entre 1 e 10000.

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 2; i < 10000; i++) {
        if (ehPrimo(i)) {
            cout << i << "\n";
        }
    }
    return 0;
}
```

4. **(Fácil)** Faça um programa que receba os valores antigo e atual de um produto. Use uma função que determine o percentual de acréscimo entre esses valores. O resultado deverá ser mostrado pelo programa principal.

Exemplo:

Entradas

10 15

```
100 110
134 134
Saídas esperadas
0.50
0.10
0.00
```

5. **(Fácil)** Faça uma função que calcule a média ponderada com pesos respectivamente 1, 2 e 3 para as provas 1, 2 e 3. Faça também outra função que decida se um aluno foi aprovado ou reprovado, sabendo que a aprovação deve ter a média final maior ou igual a 50. As notas das provas e a média são valores inteiros entre zero e 100. Você pode testar sua função usando o código abaixo:

Exemplo de entrada

```
3
100 40 5
5 40 100
50 50 50
```

Saída esperada

```
aluno 1 reprovado com media: 32
aluno 2 aprovado com media: 64
aluno 3 aprovado com media: 50
```

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n, p1, p2, p3;
    for (int i = 0; i < n; i++) {
        cin >> p1 >> p2 >> p3;
        media = media_ponderada(p1, p2, p3);
        if (aprovado(media)) {
            cout << "Aluno " << i + 1 << "aprovado com media: " << media << "\n";
        } else {
            cout << "Aluno " << i + 1 << "reprovado com media: " << media << "\n";
        }
    }
    return 0;
}
```

Vetores

1. **(Fácil)** Faça um programa que leia um número inteiro n no intervalo $[0, 200]$ e em seguida leia uma sequência de n valores reais e os insira em um vetor de reais. O programa deve imprimir na saída o resultado da divisão da soma dos valores positivos que estão em posições pares pela soma dos valores negativos que estão nas posições ímpares. Cuidado com divisões por zero.

Exemplo de entrada 1

4

-2.0 -7.0 7.0 2.0

Saída esperada 1:

-1.00

Exemplo de entrada 2

3

1 2 3

Saída esperada 2:

divisao por zero

Exemplo de entrada 3

0

Saída esperada 3:

vetor vazio

2. **(Fácil)** Faça um programa que leia uma sequência de inteiros terminada em zero e armazene esta sequência em um vetor. O zero não deve ser processado pois serve para marcar o final da entrada de dados. Em seguida o programa deve ler uma outra sequência de inteiros também terminada em zero e, para cada valor lido, o programa deve dizer qual a posição do valor dentro do vetor, caso o valor exista no vetor, ou -1 caso não exista. Esta segunda sequência não precisa ser armazenada em vetores. Use ao máximo funções.

Exemplo de entrada

-2 -6 7 2 0

7

3

2

0

Saída esperada

2

-1
3

3. **(Médio)** Faça um programa que leia um inteiro positivo n e em seguida leia n valores inteiros quaisquer e imprima "sim" se o vetor estiver ordenado em ordem crescente e "nao" em caso contrário. Em qualquer caso, o programa também deve imprimir o vetor, mas na ordem inversa com relação à leitura. Use funções e procedimentos apropriados.

Exemplo de entrada 1

5 -2 -7 7 2 1

Saída esperada 1

nao

1 2 7 -7 -2

Exemplo de entrada 2

7

1 3 4 8 8 10 15

Saída esperada 2

sim

15 10 8 8 4 3 1

Exemplo de entrada 3

0

Saída esperada 3

vetor vazio

4. **(Médio)** Faça um programa que leia uma sequência de códigos de operação e valor, onde o código de operação é um inteiro com os seguintes valores:

0 (zero): fim 1 (um): inserção 2 (dois): remoção

O valor lido é um número real que deve ser inserido em um vetor (caso a operação seja 1), ou removido do vetor (caso a operação seja 2). As inserções no vetor devem ser realizadas de forma que o vetor esteja sempre ordenado. O programa deve imprimir todos os vetores resultantes de cada operação e ao final deve imprimir o vetor resultante. Imprima os valores reais sempre com uma casa decimal. Caso o vetor esteja vazio, imprima a mensagem "vazio".

Detalhamento:

A quantidade máxima de valores que pode ser inserida é 200;

Se a quantidade máxima for ultrapassada o programa deve dar uma mensagem de erro (imprima a mensagem "erro");

Se for requisitada a remoção de um número não existente o programa deve dar uma mensagem de erro (imprima "erro");

Se o código de operação for inválido o programa deve continuar lendo um novo código até que ele seja 0 (zero), 1 (um) ou 2 (dois).

Use ao máximo funções e procedimentos apropriados.

Exemplo:

Entrada:

1

45.3

1

34.3

4

9

1

40.8

2

11.9

2

34.3

0

Saída:

45.3

34.3 45.3

34.3 40.8 45.3

erro

40.8 45.3

40.8 45.3

5. **(Difícil)** Faça um programa que leia um número inteiro positivo n e em seguida leia uma sequência de n números inteiros quaisquer. Seu programa deve imprimir os números distintos (na mesma ordem relativa entre eles) e em seguida o número de vezes que cada um deles ocorre na mesma. Use ao máximo funções e procedimentos adequados. Dica:

você pode usar vetores auxiliares para ajudar a verificar e contar as ocorrências.

Exemplo de entrada 5

3 1 2 2 3

Saída esperada:

a sequência tem 3 números distintos: 3 1 2

3 ocorre 2 vezes

1 ocorre 1 vez

2 ocorre 2 vezes

6. **(Médio)** Faça um programa que leia diversas linhas nas quais as linhas ímpares contêm o tamanho n ($0 \leq n \leq 100$) da sequência de números inteiros que deve ser lida na linha subsequente. Quando uma linha ímpar contiver o valor zero significa que a entrada de dados acabou e seu programa deverá encerrar com a impressão de todas as sequências originais e também compactadas pela eliminação de todos os elementos repetidos de cada sequência. No exemplo abaixo, O significa a sequência original e C a sequência compactada.

Exemplo de entrada:

5

2 4 7 -1 2

3

1 1 1

7

3 4 5 3 7 5 1

0

Saída esperada:

O: 2 4 7 -1 2

C: 2 4 7 -1

O: 1 1 1

C: 1

O: 3 4 5 3 7 5 1

C: 3 4 5 7 1

7. **(Médio)** Fazer um programa que leia do teclado dois números naturais $0 < n, m \leq 100$. Em seguida, leia duas sequências de respectivamente n e m números também naturais, sendo garantidamente $m < n$. Seu programa deve determinar quantas vezes a segunda sequência ocorre na primeira.

Exemplo de entrada 1:

7
2
18 23 6 14 6 31 13
12 20

Saída esperada:

0

Exemplo de entrada 2:

9
4
18 23 6 14 6 18 23 6 14
18 23 6 14

Saída esperada:

2

Exemplo de entrada 3:

30
3
2 4 2 1 6 1 7 2 9 10 2 4 2 1 8 11 12 13 2 7 1 5 6 1 3 2 4 2 4 2
2 4 2

Saída esperada:

4

8. **(Difícil)** Faça um programa que leia um número natural $0 < n \leq 100$ e em seguida leia uma sequência de n números também naturais. Seu programa deve verificar se existem duas subsequências iguais nesta sequência com tamanho pelo menos 2. O tamanho da sequência encontrada deverá ser máximo, se ela existir. Caso exista, seu programa deve imprimir o valor do índice i e do tamanho máximo da sequência m , nesta ordem, onde i é a primeira ocorrência da sequência que possui uma cópia na sequência original e m é o tamanho desta sequência que se repete. Caso contrário seu programa deve imprimir "nenhuma".

Os casos de teste não conterão entradas com mais de uma subsequência igual.

Exemplo de entrada 1:

8
7 9 5 4 5 5 4 6

Saída esperada para a entrada acima:

3 2

Exemplo de entrada 2:

12

2 7 9 5 2 5 4 8 6 2 5 4

Saída esperada para a entrada acima:

5 3

Sugestão: Use como base para sua implementação a seguinte estrutura inicial de programa, que contém o programa principal e algumas funções e procedimentos que visam facilitar o seu trabalho. Evidentemente você pode ignorar esta sugestão. Caso aceite, você deve implementar as funções e procedimentos, o programa principal não deveria ter que ser alterado, a princípio. Você pode também decidir usar mais funções e procedimentos caso perceba que seu programa ficará mais legível.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    int v[100];

    cin >> n;
    ler_vetor(v, n);

    int pos = 0;
    int tamanho_subsequencia = n / 2;

    while (pos == 0 && tamanho_subsequencia >= 2) {
        pos = tem_subsequencias_iguais(v, n, tamanho_subsequencia);
        if (pos == 0) {
            tamanho_subsequencia--;
        }
    }

    if (pos > 0) {
        cout << pos << " " << tamanho_subsequencia << endl;
    } else {
        cout << "nenhuma" << endl;
    }

    return 0;
}
```

}

Matrizes

1. **(Fácil)** Faça um programa que leia dois inteiros positivos m e n , sendo $1 \leq m, n \leq 100$, e uma matriz $A_{m \times n}$. O programa deve imprimir o número de linhas e o número de colunas que são nulas, ou seja, quando todos os elementos de uma linha ou coluna são iguais a 0 (zero).

Exemplo de entrada:

```
4 4
1 0 2 3
4 0 5 6
0 0 0 0
0 0 0 0
```

Saída esperada para o exemplo acima:

linhas: 2

colunas: 1

2. **(Difícil)** Uma matriz inteira $A_{n \times n}$ é considerada uma matriz de permutação se em cada linha e em cada coluna houver $n - 1$ elementos nulos e um único elemento igual a 1.

Um exemplo de Matriz de permutação:

```
0 1 0 0
0 0 1 0
1 0 0 0
0 0 0 1
```

Um exemplo de Matriz que não é de permutação:

```
2 -1 0
-1 2 0
0 0 1
```

Faça um programa que leia um inteiro positivo n , sendo $1 \leq n \leq 100$, e uma matriz inteira $A_{n \times n}$. O programa deve imprimir “sim” caso a matriz A seja de permutação, caso contrário deve imprimir “nao”.

PS: Seu programa deve encerrar a execução assim que descobrir se a propriedade definida foi atendida ou não.

Exemplo de entrada 1:

```
3
1 0 0
```

```
0 1 0
0 0 1
```

Saída esperada para o exemplo acima:
sim

Exemplo de entrada 2:

```
3
1 0 0
1 0 0
0 1 0
```

Saída esperada para o exemplo acima:
nao

3. **Médio** Uma matriz quadrada $A_{n \times n}$ é considerada triangular quando os elementos que estão acima da sua diagonal principal são todos nulos (matriz triangular inferior) ou quando os elementos que estão abaixo de sua diagonal principal são todos nulos (matriz triangular superior). Vejamos dois exemplos de matrizes triangulares:

Matriz Triangular Superior:

```
1 3 8 1
0 3 9 -2
0 0 1 9
0 0 0 4
```

Matriz Triangular Inferior:

```
7 0 0 0
2 1 0 0
4 9 0 0
2 1 1 8
```

Faça um programa que leia um inteiro positivo n , sendo $1 \leq n \leq 100$, e uma matriz inteira $A_{n \times n}$. O programa deve imprimir “sim” caso a matriz A seja triangular, caso contrário deve imprimir “nao”.

PS: Seu programa deve encerrar a execução assim que descobrir se a propriedade definida foi atendida ou não.

Exemplo de entrada 1:

```
3
1 9 5
0 2 4
0 0 7
```

Saída esperada para o exemplo acima:
sim

Exemplo de entrada 2:

```
3
1 0 0
5 2 0
4 9 3
```

Saída esperada para o exemplo acima:
sim

Exemplo de entrada 2:

```
3
1 2 3
4 5 6
7 8 9
```

Saída esperada para o exemplo acima:
nao

4. Faça um programa que leia dois inteiros positivos m e n , sendo $1 \leq m, n \leq 100$, e uma matriz $A_{m \times n}$. O programa deve imprimir “sim” se há elementos repetidos na matriz A , caso contrário deve imprimir “nao”.

PS: Seu programa deve encerrar a execução assim que descobrir se a propriedade definida foi atendida ou não.

Exemplo de entrada 1:

```
3 3
1 2 3
4 5 6
7 8 9
```

Saída esperada para o exemplo acima:
nao

Exemplo de entrada 2:

```
3 4
1 2 3 4
4 5 6 7
7 8 9 10
```

Saída esperada para o exemplo acima:
sim

5. (Fácil) Um vetor real X com n elementos é apresentado como resultado de um sistema de equações lineares $Ax = B$ cujos coeficientes são representados em uma matriz real $A(m \times n)$ e os lados direitos das equações (os resultados de cada equação) em um vetor real B de m elementos. Criar um programa que verifique se o vetor X é realmente solução do sistema dado, imprimindo "sim" caso o vetor informado seja solução do sistema de equações, ou imprimindo "não" caso contrário.

Dica: Comparações entre números reais pode ser um problema dependendo da arquitetura da máquina a ser utilizada. Para solucionar este problema um método a ser utilizado é a comparação por margem de erro.

PS: O sistema apresentado no exemplo de entrada é representado pela seguinte equação.

$$\begin{cases} 2x + y - 3z = -1 \\ -x + 3y + 2z = 12 \\ 3x + y - 3z = 0 \end{cases}$$

Exemplo de entrada:

3 3 tamanho da matriz
1 3 2 resultado informado
2 1 -3 Matriz
-1 3 2
3 1 -3
-1 12 0 lado direito

Saída esperada:

sim

6. (Fácil) Dizemos que uma matriz quadrada inteira é um quadrado mágico se a soma dos elementos de cada linha, a soma dos elementos de cada coluna e a soma dos elementos das diagonais principal e secundária são todos iguais. Exemplo:

8 0 7
4 5 6
3 10 2

é um quadrado mágico pois $8+0+7 = 4+5+6 = 3+10+2 = 8+4+3 = 0+5+10 = 7+6+2 = 8+5+2 = 3+5+7 = 15$.

Crie um programa que leia um valor n representando o tamanho da matriz e leia uma matrix $A(n \times n)$ que representa o quadrado e informe

se a matriz é um quadrado mágico.

Exemplo de entrada:

```
3
8 0 7
4 5 6
3 10 2
```

Saída esperada para o exemplo acima:

sim

7. Seguindo o raciocínio do exercício sobre quadrado mágico. Crie um programa que leia um inteiro n que representa o tamanho do lado de uma matriz A e uma matriz $A(n \times n)$ que representa o quadrado e imprima quantas matrizes não triviais (isto é, não pode ser a matriz que é constituída por apenas um elemento, uma linha e uma coluna) formam um quadrado mágico a partir da matriz fornecida.

Exemplo de entrada:

```
6
2 7 6 6 7 2
9 5 1 1 5 9
4 3 8 8 3 4
4 9 2 2 9 4
3 5 7 7 5 3
8 1 6 6 1 8
```

Saída esperada para a entrada acima:

5

8. (Difícil) Um jogo de palavras cruzadas pode ser representado por uma matriz $A(n \times m)$ onde cada posição da matriz corresponde a um quadrado do jogo, sendo que 0 indica um quadrado em branco e -1 indica um quadrado preto. Colocar as numerações de início de palavras horizontais e/ou verticais nos quadrados correspondentes (substituindo os zeros), considerando que uma palavra deve ter pelo menos duas letras.

PS: A imagem abaixo representa visualmente a matriz de entrada do exemplo. As palavras sempre são adicionadas da esquerda para a direita (Horizontal) ou de cima para baixo (Vertical).

1		2			3		4
5	6	0	0		7	0	0
8	0			9	0		0
	10	0	11	0		12	0
13	0		14	0	0		

Exemplo de entrada:

5 8

0 -1 0 -1 -1 0 -1 0

0 0 0 0 -1 0 0 0

0 0 -1 -1 0 0 -1 0

-1 0 0 0 0 -1 0 0

0 0 -1 0 0 0 -1 -1

Saída esperada para a entrada anterior:

1 -1 2 -1 -1 3 -1 4

5 6 0 0 -1 7 0 0

8 0 -1 -1 9 0 -1 0

-1 10 0 11 0 -1 12 0

13 0 -1 14 0 0 -1 -1